

MIPster Cut and Heuristic Orchestration

MIPster Development Team

June 2026

Contents

1	Overview	3
2	Root vs. tree control loop	3
3	The diminishing-returns / ifMove logic	4
4	Per-generator throttling	5
5	Serial vs. parallel cuts	6
6	Global cut pool and ranking	7
7	Current settings (as built)	10
8	Alternative options	11
9	Cut-selection thresholds	11
10	CbcRowCuts vs. CoinCutPool	14
11	Tree-side activation: parameters and hardcoded constants	15
12	The branchAndBound entry point	17
13	doHeuristicsAtRoot: pre-cut primal heuristics	17
14	applyRowCuts + resolve: cut installation	17
15	Node selection and branching	18
16	Method and data-structure maps	18
17	Tree-side cut data structures and lifecycle	20
18	Node cut restoration: the wholesale rebuild	22
19	fullScan and periodic generator re-evaluation	23
20	moreSpecialOptions_ and specialOptions_ cut-related bits	23
21	LP reoptimisation cost control — design notes	24

22 Complete parameter catalog	26
23 Putting it together	28

1 Overview

MIPster (a Cbc fork) drives a branch-and-cut search. At every node — including the root — it alternates three things in a fixed sequence: **(1)** solve / re-solve the LP relaxation, **(2)** run a battery of cut generators in batch and apply the selected cuts, **(3)** optionally run primal heuristics on the fractional solution. This document explains, with file/line pointers into `src/`, how that orchestration is implemented: how each pass decides whether to keep generating cuts (the “ifMove” / diminishing-returns logic), how individual generators are throttled across passes, the parallel-cuts execution mode, the global cut pool, and the mechanisms that rank cuts when too many are produced.

The two source files driving this are:

- `src/CbcModel.cpp` — per-pass orchestration (`solveWithCuts`), per-generator post-loop adjustment, `serialCuts`, `takeOffCuts`, and `doHeuristicsAtRoot`.
- `src/CbcThread.cpp` — `parallelCuts`.

2 Root vs. tree control loop

High level

`CbcModel::branchAndBound` (around `CbcModel.cpp:2700`) performs the root work in this order:

1. Initial LP solve and presolve setup.
2. **Pre-cut heuristics:** `doHeuristicsAtRoot()` at `CbcModel.cpp:3056` (FPump, dive conflict harvest, etc.).
3. **Root cut loop:** `solveWithCuts(cuts, maximumCutPassesAtRoot_, NULL)` at `CbcModel.cpp:3386` — up to `maximumCutPassesAtRoot_` passes of (*generate cuts* → *apply* → *resolve* → *post-pass heuristics*).
4. **Per-generator adjustment:** after the loop, every cut generator’s `howOften` is updated based on how much it contributed at the root (`CbcModel.cpp:10080–10591`).
5. Branch-and-bound proper begins; at every node `solveWithCuts(cuts, maximumCutPasses_, node)` is called from `CbcModel.cpp:17919`.

The pass loop in `solveWithCuts`

`solveWithCuts` (`CbcModel.cpp:8333+`) is the same routine used at root and at every node; the second argument (`numberTries`) gates how many passes are allowed. The loop body lives at `CbcModel.cpp:9000–9711`. Each pass:

1. Increments `currentPassNumber_` (line 9001).
2. On the very first pass, scans the global cut pool `globalCuts_` for cuts violated by the current relaxation (line 9066) and selects up to a budget by *violation-sorted* order (see §6 for the pool itself and §9 for how the element budget is computed).
3. Calls either `serialCuts` or `parallelCuts` (lines 9272–9279) to invoke every active generator.
4. Skips a primal-heuristic block on pass 1 of the root, runs it on later passes (line 9295).
5. Applies the chosen cuts via `solver_>applyRowCuts(...)` (line 9514).
6. Re-solves the LP via `resolve(...)` (line 9533).
7. Calls `takeOffCuts` (line 9582) to drop cuts that slacken (§6).

8. Applies the diminishing-returns / `ifMove` test (lines 9595–9711) and decides whether to keep looping (see §3 for the full criterion).

So the answer to “do we add X, reoptimize and then run Y?” is: **all generators run together in one batch per pass, then one resolve, then heuristics**, then back to step 3. Generators are not interleaved one-at-a-time with re-solves.

Tree-depth gating

Inside the tree, `solveWithCuts` additionally honours `treeCutDepth_` (`CbcModel.cpp:8357–8378`): below the threshold `numberTries=1` (cuts are tried), above it `numberTries=0` (only resolve). The deeper helper `doCutsNow` at `CbcModel.cpp:18885–18942` provides a finer per-depth gate used by individual generators (see §2).

`doCutsNow` per-depth gate

`CbcModel::doCutsNow(int allowForTopOfTree) const` (`CbcModel.cpp:18885–18942`) returns `true` when the current node depth and the configured `whenCuts_` mode agree that cuts should be generated here. The author’s comment at `CbcModel.cpp:18880–18883` states the four argument values:

- 0 — only on multiples of the configured depth.
- 1 — also allow “at top of tree” (`currentDepth_ <= shallow`).
- 2 — “cuts allowed anywhere apart from root”.
- 3 — “smallest valid depth > shallow”; in the currently-active `TRY_IDEA1=2` branch this collapses to “only at exactly depth 10” (`CbcModel.cpp:18934–18936`).

Internally, `whenCuts_` (set via `setWhenCuts`, exposed on the CLI as `-cuts`) is decoded into a `top/shallow/when` triple (`CbcModel.cpp:18914–18916`); for small models (≤ 500 rows+cols) and `whenCutsUse < 0` the routine falls into a simpler depth-parity branch (lines 18899–18911). This is the single place where per-generator `depthCutGenerator_` settings (§4) and the tree-wide `treeCutDepth_` (§2) interact with node depth.

3 The diminishing-returns / `ifMove` logic

The cut loop terminates early if cuts stop helping the LP bound. The relevant code is at `CbcModel.cpp:9595–9711`, and is controlled by `minimumDrop_` (default $1.0e-7$, set in `CbcModel.cpp:6001/619` tightened in `CbcSolver.cpp` to $\min(5e-2, |\text{obj}|*1e-5+1e-5)$).

The rolling history

A buffer keeps the last `CUT_HISTORY=7` pass objectives (`CbcModel.cpp:8381`). Two flags steer the test:

```
int   experimentBreak = (moreSpecialOptions_ >> 11) & 3;    // 9969
bool  increaseDrop    = (moreSpecialOptions_ & 8192) != 0;   // 9971
```

The `badObj` / `goodDrop` test

After the resolve, the loop compares the new bound against the oldest one in the history window:

```
bool badObj = thisObj < cut_obj[CUT_HISTORY-1] + minimumDrop_;
double goodDrop = thisObj - cut_obj[0];
// nBadPasses incremented if drop is too small
```

```
if (badObj && allowEarlyTermination && !keepGoing)
    numberTries = 0;           // exit the loop
```

The loop is allowed to exit when:

- `badObj` fires (no progress vs. the 7-pass window), *and*
- no generator flagged itself as “must call again” (`keepGoing`), *and*
- the early-termination guard is on (`allowEarlyTermination`).

If `increaseDrop` is set, `minimumDrop_` ramps up over passes, making the loop bail out faster. `experimentBreak` toggles two stricter variants of the test (used for tuning runs).

Per-pass “ifMove” for individual cuts

Beyond the loop-level test, individual cuts can be marked “ifMove” through their generator’s mode. This translates into an effectiveness threshold passed to `takeOffCuts`: cuts whose effectiveness is below the threshold and whose row is basic in the current basis are dropped (§6). In other words, “ifMove” = “keep the cut only if it is still binding / effective after the resolve.”

4 Per-generator throttling

Each `CbcCutGenerator` carries:

- `whenCutGenerator_` (a.k.a. *howOften*): 0 = off, 1 = every pass, -1 = root only, -99 = “almost off (only when the search is desperate)”, -100 = permanently off.
- `depthCutGenerator_`: depth multiplier (run only when *depth* mod *n* = 0).
- `switchOffIfLessThan_`: yield threshold below which the generator is shut off post-root.
- A `switches_` bitfield with 16 flags, including `normal`, `atSolution`, `whenInfeasible`, `mustCallAgain`, `switchedOff`, `ineffectualCuts`, `whetherToUse`, `whetherInMustCallAgainMode`, `whetherCallAtEnd`, `needsRefresh`, `cleanLagrangian`. Definitions in `src/CbcCutGenerator.hpp` (lines covering the bit constants).

The per-call gate is `CbcCutGenerator::generateCuts` (`src/CbcCutGenerator.cpp`:186-262). Simplified:

```
int howOften = whenCutGenerator_;
if (howOften <= -100 || howOften == 0) return false;           // off
int pass = model_>getCurrentPassNumber() - 1;
bool doThis = (model_>getNodeCount() % howOften) == 0;
if (depthCutGenerator_ > 0)
    doThis = (currentDepth % depthCutGenerator_) == 0;
if (whenCutGeneratorInSub_ == -200 && model_>parentModel())
    doThis = false;                                           // off in subtree
```

Once cuts are produced, the per-call exit also flips the generator’s `ineffectualCuts` bit if no cut was retained — this feeds the post-loop adjustment described next.

Post-loop `howOften` decay (root only)

After the root cut loop, `CbcModel.cpp`:10080-10591 loops over the generators and rewrites `howOften` based on how much they actually contributed. Highlights:

```
// 10140 -- if no objective change and rows grew >20%
if (numberRowsAtContinuous_ * 1.2 < numberRowsAtStart
```

```

    && noObjectiveChange)
willBeCutsInTree = -1;

// 10419 -- generator-specific shut off
if (howOften == -98 && switchOffIfLessThan > 0
    && objImprovement < 0.005 * fabs(startObj) + 1e-5)
    howOften = -99;

// 10474..10534 -- rebuild howOften from cut yield
//   no cuts      -> howOften = -99 (almost off)
//   few cuts     -> howOften = max(2, ...)
//   many cuts    -> howOften = 1 (every pass)

// 10578
generator_[i]->setHowOften(howOften);

```

The “ifMove” parameter style (`-cuts ifmove`, `-gomory ifmove`, etc.) ultimately sets `howOften` to `-98` for the corresponding generator, which causes the `switchOffIfLessThan_` test above to demote it to `-99` unless it pulled its weight at the root.

Important interaction with pass-0 early disable. Generators with `switchOffIfLessThan_ > 0` are also disabled *during* root cutting: at the end of pass 0 (`CbcCutGenerator.cpp:1583-1591`), if the generator produced fewer row cuts than the threshold, `whenCutGenerator_` is set directly to `-100`. The post-loop then reads `howOften = -100` at the start of its per-generator iteration, skips the adjustment logic, and *only emits a `CBC_GENERATOR` summary message if the generator found at least one cut overall*. This means `TwoMirCuts` (`switchOffIfLessThan = 1`) and `ZeroHalf` (`switchOffIfLessThan = 2`) may be silently absent from the cut summary on problems where they find nothing — even though they were configured and ran on pass 0. The fix applied in MIPster removes the “ $n > 0$ ” guard so all generators are always reported at root (`CbcModel.cpp:10757-10774`).

Generator scan caps

Two compile-time caps live in `src/CbcCutGenerator.hpp`:

```

#define SCANCUTS          1000
#define SCANCUTS_PROBING 1000

```

These bound the number of rows / columns a generator scans per call (generators that respect them stop early on huge models).

5 Serial vs. parallel cuts

Two paths exist for invoking the generators in a pass.

`serialCuts` (default)

`CbcModel::serialCuts` (`CbcModel.cpp:10797-10915`) walks `generator_[0 .. numberCutGenerators_-1]` in a single thread, calling `generateCuts` on each one that is `normal()`, not `switchedOff()`, satisfies its `needsOptimalBasis()` requirement, and matches the current `whetherCallAtEnd()` phase. Cuts produced are merged into the local `theseCuts` pool.

`parallelCuts`

`CbcModel::parallelCuts` (`src/CbcThread.cpp:1954`) runs each generator in its own thread via `master->waitForThreadsInCuts(...)`, then merges the results. It honours `mustCallAgain`

(sets the loop’s status to “keep going”).

The selection happens at `CbcModel.cpp:9272-9279`:

```
if ((threadMode_ & 2) == 0 || numberNodes_)
    serialCuts(theseCuts, node, fullScan, eachCutLimit);
else
    parallelCuts(master_, theseCuts, node, eachCutLimit);
```

So parallel cuts are used *only at the root* (`numberNodes_==0`) and only when bit 2 of `threadMode_` is set. Inside the tree, threading is used for parallel *node* expansion (bit 1) and parallel heuristics (bit 4) instead.

6 Global cut pool and ranking

Yes, it is a global pool

There is exactly one cut pool per `CbcModel`, declared at `CbcModel.hpp:3028`:

```
CbcRowCuts globalCuts_;
```

It is “global” in the sense that *any* node can deposit cuts into it and *every* subsequent node may pick them up via the pass-1 re-scan in `solveWithCuts` (`CbcModel.cpp:9066-9131`). Cuts placed here must therefore be **globally valid** — valid for the whole MIP, not just the sub-problem at one node. Generators (and `takeOffCuts`) mark cuts globally valid via `OsiRowCut::setGloballyValid()` or `setGloballyValidAsInteger(2)` before insertion (e.g. `CbcModel.cpp:3336, 3584, 4893-4905, 8509, 9042, 9802`).

Internal structure: hash-deduplicated array

`CbcRowCuts` (`src/CbcCountRowCut.cpp:383-660`) is a dynamic array of `OsiRowCut2 * (rowCut_[])` backed by a separately-chained hash table (`hash_[]`, sized `hashMultiplier_ * size_`). The flow inside `addCutIfNotDuplicate` (line 587) is:

1. If the array is at capacity, double it (`size_ = 2 * size_ + 100` at line 595) and rebuild the hash table from scratch. So the pool is in practice unbounded; “hash full” only happens transiently during the rebuild and is handled by widening the cut’s bounds rather than dropping it (lines 626-635).
2. Compute the cut’s hash (`hashCut(cut, hashSize)`) — a function of the sorted index/coefficient pattern.
3. Walk the chain at that bucket; if a cut with the same `same(cutA, cutB)` signature is found, return 1 (“not added, duplicate”).
4. Otherwise sort the cut’s indices, sanity-check coefficient magnitudes ($10^{-12} \leq |a_i| \leq 10^{12}$), append it to `rowCut_` and link it into the hash chain. Return 0.
5. Return value `-1` is reserved for “no space” and is never actually returned in practice (the auto-resize covers it).

So the pool is effectively a **hash-set of globally valid row cuts with insertion-order preservation**. Lookup is $O(1)$ average, insertion is $O(1)$ amortised, and there is no built-in eviction policy beyond manual `truncate(n)` (line 524) and `eraseRowCut(seq)` (line 465). See §10 for how this differs from the ranking-aware `CoinCutPool` used inside individual cut generators.

Who deposits into the pool

Cuts arrive in `globalCuts_` from many places — it is the solver’s persistence mechanism for everything that should survive a node:

Site	Cuts deposited
<code>CbcModel.cpp:3226–3337</code>	root racing across multiple solvers (<code>multipleRootSolvers</code>)
<code>CbcModel.cpp:3584–3586</code>	strong-branching cuts that became globally valid
<code>CbcModel.cpp:4893–4905</code>	integer-feasible cuts derived from incumbents
<code>CbcModel.cpp:8509</code>	<code>makeGlobalCut</code> — the canonical entry point used by anyone
<code>CbcModel.cpp:9042</code>	cuts marked globally valid during <code>solveWithCuts</code>
<code>CbcModel.cpp:9802</code>	post-pass promotion of generator cuts
<code>CbcModel.cpp:11214–11223</code>	<code>takeOffCuts</code> parks slack cuts back here (§6)
<code>CglStored, CglDuplicateRow</code>	purpose-built persistent generators

Who consumes the pool

- Pass 1 of `solveWithCuts`: the violation-sorted re-scan with element budget (§9).
- `CbcModel::resolve` indirectly — once a cut has been promoted into the LP it is just another row.

Per-cut effectiveness

Every `OsiRowCut` carries an `effectiveness()` value (`double`) used both for purging (§6) and for ranking when too many cuts compete for a budget. `Cgl` generators set it as follows:

Generator	Effectiveness	Source
MIR / MIR2	violation	<code>CglMixedIntegerRounding[2].cpp:1484/1716</code>
LandP	violation	<code>CglLandPSimplex.cpp:594</code>
ResidualCapacity	violation	<code>CglResidualCapacity.cpp:626</code>
Stored cuts	violation	<code>CglStored.cpp:67–152</code>
KnapsackCover	<code>COIN_DBL_MAX</code>	<code>CglKnapsackCover.cpp:1091/1121</code>
AllDifferent	100	<code>CglAllDifferent.cpp:397</code>
DuplicateRow	100	<code>CglDuplicateRow.cpp:881/2262/3107</code>
Saved-to-slack sentinel	−1.234	<code>CbcModel::takeOffCuts</code>

`COIN_DBL_MAX` means “always keep”; the −1.234 sentinel flags a cut that has been parked on the slack pool to be considered again later.

Ranking site 1: global pool re-scan (pass 1)

When pass 1 of `solveWithCuts` scans the global pool for cuts violated by the current LP, far more may be violated than the relaxation can absorb without bloating. The code at `CbcModel.cpp:9363` scores each violated cut and then sorts descending, applying a matrix-element budget:

```
// Score depends on -cutRankingMetric:
// violation (0): score = thisCut->violated(x*)
// fitness (1): score = CoinCutPool::rowCutFitness(...)
CoinSort_2(scores, scores + numberPossible, which);
int maximumAdd =
    std::max(numberElements / 10, 2 * numberColumns) + 100;
// pick from 'which' until maximumAdd nonzeros consumed
```


So the rule is: *best-scoring first (violation or fitness), until the matrix-element budget is exhausted*. Cuts with `effectiveness() = COIN_DBL_MAX` bypass the budget.

The global-pool admission threshold (minimum violation to even consider a cut) is controlled by `-globalCutMinViolation` (default 0.005, field `globalCutMinViolation_`).

Per-round cut pool filter (`filterCutsByBestPerVar`)

After each cut-generation pass but before LP insertion, if `-cutPoolFilter` is on (default) and the round produced at least `-cutPoolFilterMinCuts` row cuts (default 50), the static helper `filterCutsByBestPerVar` (`CbcModel.cpp:8530`) is applied.

The algorithm replicates the `CoinCutPool` best-by-variable logic (§10) directly on the `OsiCuts` batch:

1. Compute the fitness score for every row cut using `CoinCutPool::rowCutFitness`.
2. For each column, record the index of the highest-fitness cut that contains it.
3. A cut survives only if it is the best cut for *at least one* column. Cuts that are strictly dominated in all their columns are removed.
4. Column cuts are always kept unchanged.

The filter is a no-op when fewer than 2 row cuts are present. On instances with many cuts per round (e.g. CMS750_4 produces ~ 1473 cuts per round) this can significantly reduce LP bloat. On instances with few cuts per round (e.g. d05100, ~ 2 cuts) the `-cutPoolFilterMinCuts` threshold skips the filter and avoids any overhead.

```
// CbcModel.cpp:9654-9657
if (cutPoolFilter_ && theseCuts.sizeRowCuts() >= cutPoolFilterMinCuts_)
    filterCutsByBestPerVar(theseCuts, cbcColSolution_, getNumCols());
```

Fitness score formula

All three ranking sites and the round filter share the same fitness formula, exposed as the static method `CoinCutPool::rowCutFitness` (`src/CoinUtils/CoinCutPool.cpp`):

$$\text{fitness}(C) = \frac{\text{viol}(C)}{|\{i : x_i^* > 0\}|} \cdot 10^5 + \frac{1}{\Delta_C + 1} \cdot 10^3$$

where Δ_C is the spread of coefficient magnitudes ($\log_{10}(\max |a_i|) - \log_{10}(\min |a_i|)$). This favours cuts that are **(a)** violated relative to the number of active variables (sparse, targeted) and **(b)** numerically balanced (small coefficient spread). The fitness metric is the default (`-cutRankingMetric fitness`).

For $\geq$ -form cuts (lower-bound only), the formula is applied after negating coefficients and right-hand side to bring the cut into $\leq$ -form. Cuts with zero active columns fall back to raw violation.

Ranking site 2: dive conflict cuts

`doHeuristicsAtRoot` (the MIPster path, `CbcModel.cpp:17160-17185`) collects conflict cuts from the `CbcRootHeuristicSchedule` dives. It scores each cut using the active `-cutRankingMetric` (violation or fitness) and keeps the top `conflictMaxCuts()`:

```
// fitness metric (default):
scored[i] = { CoinCutPool::rowCutFitness(...), i };
// violation metric:
```

```
scored[i] = { (lhs - c.ub()) / c.row().getNumElements(), i };
std::sort(scored.begin(), scored.end(), descending);
nAdded = std::min(nTotal, maxCuts);
```

Ranking site 3: takeOffCuts purge

`CbcModel::takeOffCuts` (`src/CbcModel.cpp:11164-11349`) runs after every resolve. For each cut just applied, it inspects the warm-start basis status:

- If the cut’s row is *basic* (i.e. slack is in the basis $\Rightarrow$ the cut is not binding) *and* `effectiveness()` $\leq 1e10$, the cut is removed from the LP and parked in the `slackCuts` pool with `setEffectiveness(-1.234)`
- Cuts with `effectiveness() = COIN_DBL_MAX` (e.g. knapsack covers) are kept regardless of basis status.

This is the mechanism that keeps the LP from growing unboundedly across passes and which gives the “ifMove” modes their real teeth: a cut that does not bite is gone by the next pass.

7 Current settings (as built)

CbcModel constructor defaults

```
maximumCutPassesAtRoot_ = 20;      // CbcModel.cpp:6087
maximumCutPasses_       = 10;      // CbcModel.cpp:6088
minimumDrop_            = 1.0e-7;  // CbcModel.cpp:6001/6191
```

CbcSolver adaptive overrides

If the user does not pass `-cutPasses` / `-passC!uts`, `CbcSolver.cpp:10072-10093` chooses based on column count:

```
if (cutPass == -1234567) {                // user-default sentinel
    if (numCols < 500)    setMaximumCutPassesAtRoot(-100);
    else if (numCols < 5000) setMaximumCutPassesAtRoot(100);
    else                 setMaximumCutPassesAtRoot(50);
} else {
    setMaximumCutPassesAtRoot(cutPass);
}

if (cutPassInTree == -1234567) setMaximumCutPasses(4);
else                           setMaximumCutPasses(cutPassInTree);

setMinimumDrop( std::min(5.0e-2, std::fabs(obj)*1.0e-5 + 1.0e-5) );
```

A negative `maximumCutPassesAtRoot_` (e.g. `-100`) means “loop until cuts stop helping” (the loop relies entirely on the ifMove escape from §3).

CLI defaults (CbcParameters.cpp)

Parameter	Default	Where
<code>cutPass!es</code>	100	line 965
<code>passC!uts</code> (in tree)	1	line 838
<code>slow!cutpasses</code>	10	line 2470

Default generator and heuristic order

Cut generators are registered in this order in `src/CbcSolverCutSetup.cpp`: Probing, Gomory, KnapsackCover, RedSplit, Clique (BK), OddWheel, MIR2, FlowCover, PathAggregation, TwoMirCuts, ZeroHalf. Primal heuristics are registered at `CbcSolver.cpp:9298-9322`: FPump, Rounding, Local (combine), GreedyCover, GreedyEquality (optionally RandRound).

8 Alternative options

Pass / drop control

Parameter	Effect
<code>-cutPass!es N</code>	root pass cap (§7)
<code>-passC!uts N</code>	per-node pass cap
<code>-slow!cutpasses N</code>	cap on “slow” cut passes (those flagged via <code>maximumTries</code>)
<code>-cuts ifmove</code>	set every generator to “keep only if it moved the LP” (howOften)
<code>-cuts on / root / ifmove / forceOn / off</code>	global override of every generator’s howOften
<code>-X ifmove</code> (per generator)	same, but only for X (<code>-gomory</code> , <code>-twomir</code> , <code>-knapsack</code> , <code>-clique</code>)

Threading

`threadMode_` is a bitfield set via `-threads N` and `-threadMode M`:

Bit	Meaning
1	parallel B&B node expansion
2	parallel root cuts (this section’s <code>parallelCuts</code>)
4	parallel root heuristics

Other relevant tuning

Parameter	Effect
<code>-multipleRootSolvers aabbcc</code>	race several LP/cut configs at the root (<code>CbcParameters.cpp:2488</code>)
<code>-cutDepth N</code>	also try cuts at depths $0, N, 2N, \dots$
<code>-treeCutDepth N</code>	disable cuts below depth N in the tree (§2)
<code>-conflictCuts on / off</code>	toggle dive conflict-cut harvest

9 Cut-selection thresholds

When “too many cuts” are produced, the solver does *not* keep a fixed top- K subset before inserting them in the LP. Instead it applies a sequence of **admission filters** and a post-resolve **purge**. In other words: *insert (almost) everything, re-solve, then drop what did not bind*.

1. Per-cut admission threshold (global pool re-scan)

When the global pool is consulted at the start of a pass (`CbcModel.cpp:9348-9400`), each saved cut is admitted only if its violation against the current x^* exceeds the `-globalCutMinViolation` threshold (default 0.005):

```
double violation = thisCut->violated(cbcColSolution_);
if (violation > globalCutMinViolation_) { // default 0.005
    // score and collect for ranking
}
```

Cuts with smaller violation are skipped entirely. This threshold is tunable via `-globalCutMinViolation`.

2. Element budget on global-pool re-injection

After the per-cut filter, the surviving cuts are sorted by descending violation (§6) and admitted one-by-one until a **matrix-element budget** is exhausted (`CbcModel.cpp:9075`):

```
CoinBigIndex maximumAdd =
    std::max( numberElements / 10,
              static_cast<CoinBigIndex>(2 * numberColumns) ) + 100;
// ...
if (violations[i] != -COIN_DBL_MAX)
    maximumAdd -= thisCut->row().getNumElements();
if (maximumAdd < 0) break;           // budget exhausted
```

The budget is therefore $\max(nz/10, 2n) + 100$ nonzeros, where nz is the current LP's nonzero count and n the column count. Cuts with effectiveness = `COIN_DBL_MAX` bypass the budget (they never decrement `maximumAdd`).

3. Re-admitting slack cuts

When a pass produces no fresh cuts but the slack-cuts pool `slackCuts` (populated by `takeOffCuts`, §6) contains cuts that have become violated again, they are re-admitted using a tighter primal-tolerance test (`CbcModel.cpp:11109`):

```
if (thisCut->violated(cbcColSolution_) > 100.0 * primalTolerance)
    theseCuts.insert(*thisCut);
```

4. Per-generator caps inside `generateCuts`

Each generator can be silenced or throttled before its cuts even reach the LP. The relevant fields live in `src/CbcCutGenerator.hpp` (constructor defaults in `CbcCutGenerator.cpp:56,79`):

Field	Default	Effect when triggered
<code>switchOffIfLessThan_</code>	0	kill generator on its first call if it produced fewer cuts than this (<code>CbcCutGenerator.cpp:1583</code>)
<code>maximumTries_</code>	-1 (off)	hard-cap on how many calls the generator gets (<code>CbcCutGenerator.cpp:1583</code>)
<code>whenCutGenerator_</code>	-	frequency (§4)

The relevant snippet:

```
// CbcCutGenerator.cpp:1583
if (node == NULL && !pass) {
    if (numberRowCutsAfter - numberRowCutsBefore < switchOffIfLessThan_) {
        maximumTries_ = 0;
        whenCutGenerator_ = -100;           // permanently off
    }
}
if (maximumTries_ > 0) {
    maximumTries_--;
    if (!maximumTries_) whenCutGenerator_ = -100;
}
```

The values actually assigned by `installCutGenerators` (`src/CbcSolverCutSetup.cpp`) via the `switches[]` array:

Generator	switchOffIfLessThan_
Probing, Gomory, Knapsack, Clique, OddWheel	0 (never early-disabled)
MIR, FlowCover, PathAggregation	0
TwoMirCuts	1
ZeroHalf	2
RedSplit, LandP	uses <code>maximumTries_</code> instead

TwoMirCuts and ZeroHalf will therefore be permanently disabled after pass 0 if they produce no (or fewer than threshold) cuts, and will only appear in the root summary if they found at least one cut — see the note in §4 on the post-loop decay.

5. Per-cut density cap (Gomory)

The CLI parameter `-cutLength` (`CbcParam::CUTLENGTH`, default `-1` = unbounded) is plumbed through to `CglGomory::setLimit{,AtRoot}` in `src/CbcSolverCutSetup.cpp:175-181`:

```
int cutLength = parameters[CbcParam::CUTLENGTH]->intVal();
if (cutLength != -1) {
    gomoryGen.setLimitAtRoot(cutLength);
    if (cutLength < 10000000) gomoryGen.setLimit(cutLength);
    else
        gomoryGen.setLimit(cutLength % 10000000);
}
```

`setLimit` causes `CglGomory` to reject any cut whose support exceeds `cutLength` variables — a per-cut filter applied *inside* the generator before it ever reaches `theseCuts`.

6. Dive conflict cuts: top- K violation sort

The single place where a hard “keep top K cuts, drop the rest” rule *is* applied is the dive conflict-cut harvest at `CbcModel.cpp:9970-10008` and `16800-16846`:

```
int maxCuts = schedule.conflictMaxCuts(); // default 128
int nTotal = cuts.sizeRowCuts();
if (nTotal <= maxCuts) {
    // insert all
} else {
    // score by (lhs - ub) / nElements (normalised violation)
    std::sort(scored.begin(), scored.end(), descending);
    nAdded = std::min(nTotal, maxCuts);
    for (int i = 0; i < nAdded; i++)
        solver_>applyRowCuts(1, &cuts.rowCut(scored[i].second));
}
```

The cap is `conflictMaxCuts_ = 128` (default in `src/CbcRootHeuristicSchedule.cpp:26`, settable via `CbcRootHeuristicSchedule::setConflictMaxCuts`). Cuts beyond the 128th most violated (per nonzero) are silently discarded.

7. Cut-pool capacity

`CbcRowCuts` `globalCuts_` has a fixed-size hash table; its `addCutIfNotDuplicate` returns `-1` when the table is full and the new cut is dropped (no preference among existing entries is expressed — old cuts are kept, the new one loses). Use `CbcRowCuts::truncate(n)` (line 524) to reduce the pool manually if a custom strategy needs to make room.

8. Post-resolve purge (`takeOffCuts`)

After `applyRowCuts` and `resolve`, *every* generated cut is re-evaluated by `takeOffCuts` (§6). A cut whose row is basic in the new basis *and* whose `effectiveness()` is finite ($\leq 10^{10}$) is removed from the LP and parked in the slack-cuts pool. This is the de-facto answer to “how does the solver decide which cuts to keep?”: insert what passed the admission filters above, then keep only the cuts that actually bound the next LP relaxation.

Summary of thresholds

Threshold	Where	Default
Global-pool violation cut-off	<code>CbcModel.cpp:9363</code> (<code>globalCutMinViolation_</code>)	0.005 (CLI: <code>-globalCutMinViolation</code>)
Global-pool element budget	<code>CbcModel.cpp:9342</code>	$\max(\text{nz}/10, 2n) + 100$
Slack-cut re-admit cutoff	<code>CbcModel.cpp:11109</code>	$100 \tau_{\text{prim}}$
<code>switchOffIfLessThan_</code>	<code>CbcCutGenerator.cpp:1583</code>	0 per generator (<code>TwoMir=1, ZH=2</code>)
<code>maximumTries_</code>	<code>CbcCutGenerator.cpp:1593</code>	-1 (off)
Gomory cut-length cap	<code>CbcSolverCutSetup.cpp:175</code>	-1 (off)
Dive conflict-cut top- K	<code>CbcModel.cpp:10288</code>	$K = 128$
Pool capacity (hash full)	<code>CbcCountRowCut.cpp:587</code>	implementation-defined
<code>takeOffCuts</code> drop	<code>CbcModel.cpp:11214</code>	$\text{basic} \wedge \text{effectiveness} \leq 10^{10}$
Cut pool filter threshold	<code>CbcModel.cpp:9656</code>	50 (CLI: <code>-cutPoolFilterMinCuts</code>)

10 CbcRowCuts vs. CoinCutPool

MIPster has *two* different “cut pool” classes that play complementary roles. They should not be confused.

CbcRowCuts (`globalCuts_`)

The “global” pool described in §6. Job: keep a deduplicated, persistent collection of *globally valid* cuts that any node can re-use. Filter criterion: **exact duplicate** only — two cuts with the same indices, coefficients and right-hand side hash to the same bucket and are coalesced. No notion of cut quality, dominance or coverage. Selection-on-replay is delegated to the violation-sorted re-scan in `solveWithCuts` (§9).

CoinCutPool (`src/CoinUtils/CoinCutPool.{hpp,cpp}`)

A short-lived, per-call ranking-aware filter authored for the clique generators. Its constructor takes the LP solution x^* and the column count, and its `add(idxs, coefs, nz, rhs)` performs three additional filters on top of duplicate detection:

1. **Per-cut fitness** (`calculateFitness`, `CoinCutPool.cpp:223–255`):

$$\text{fit}(C) = \frac{\text{viol}(C)}{|\{i : x_i^* > 0\}|} \cdot 10^5 + \frac{1}{\Delta_C + 1} \cdot 10^3$$

where Δ_C measures the spread of coefficient magnitudes in C . This favours cuts that are violated by the current x^* *and* that are not too unbalanced numerically.

2. **Best-by-variable filter** (`updateCutFrequency`, `CoinCutPool.cpp:185-221`): for each variable i , the pool tracks the index of the cut with the largest fitness that contains i . A new cut is kept only if it is the *best* cut for at least one variable, otherwise it is discarded. This guarantees “coverage” — the pool retains a small, well-spread set instead of many near-duplicates.
3. **Dominance pruning** (`removeDominated`, `CoinCutPool.cpp:268-330`): an $O(n^2)$ post-pass that drops cuts dominated by others (`CoinCut::dominates`, line 140). Sound only for $\leq$ -cuts with non-negative coefficients (the domination test normalises by the right-hand side).

Where it is currently used

- `CglBKClique::insertCuts` (`src/Cgl/CglBKClique.cpp:370`) — a clique separator can produce hundreds of clique cuts per call; `CoinCutPool` keeps the few that matter.
- `CglOddWheel::generateCuts` (`src/Cgl/CglOddWheel.cpp:121`) — same pattern.
- **filterCutsByBestPerVar** in `CbcModel.cpp:8530` — the best-by-variable criterion is now applied directly on `OsiCuts` at the round level (before LP insertion) via a static helper that replicates `CoinCutPool`’s logic without using the pool’s internal data structures. This is the per-round cut pool filter described in §6 and activated by `-cutPoolFilter` on (default).

Adoption opportunities

The remaining places in the codebase where `CoinCutPool` would replace ad-hoc top- K logic, ranked by expected impact:

1. **Global-pool re-scan in `solveWithCuts`** (`CbcModel.cpp:9340-9400`). The current logic sorts by fitness or violation but applies no coverage criterion. A `CoinCutPool` layer would add best-by-variable coverage, at the cost of $O(n_{\text{cols}})$ memory per scan.
2. **`CglCliqueStrengthening`**. Same family of cuts as `BKClique/OddWheel`; sound for the dominance test.
3. **Clique-cut path inside `CglProbing`**. Probing emits dense waves of cliques at ~ 25 call sites (`CglProbing.cpp:230, 241, 270, 1382, \dots`). Replacing those raw `cs.insert()` calls with a `CoinCutPool` would cut LP bloat substantially.
4. Legacy `CglClique` and `CglOddHole` — these ship clique cuts but predate `CoinCutPool`.

Caveats. `CoinCut::dominates` assumes a $\leq$ -cut with non-negative coefficients (it normalises by the right-hand side). It is *not* valid for Gomory, MIR, or any mixed-coefficient cut family — those generators must keep using `CbcRowCuts` or a custom filter. Also, the current `removeDominated` is $O(n^2)$ in the number of cuts in the pool; on dense probing waves this can dominate generator time, so adoption there should come with a size cap.

11 Tree-side activation: parameters and hardcoded constants

A node-level cut activation decision is the conjunction of three gating layers; understanding the layering is essential before trying to tune anything.

The three layered gates

1. **`treeCutDepth_`** (§2, default 6) — the outermost guillotine. At node depth $\geq \text{treeCutDepth_}$, `solveWithCuts` sets `numberTries=0` and skips the whole pass loop. CLI: `-treeCutDepth N`.
2. **Per-generator `whenCutGenerator_` / `depthCutGenerator_` / `switchOffIfLessThan_` / `maximumTries_`** (§4) — the per-generator policy rewritten by the post-root decay step (`CbcModel.cpp:10080-`

This is the most adaptive layer: a generator that flopped at the root is demoted to `howOften=-99` and only re-runs when the search becomes desperate.

3. **doCutsNow** (§2) — the finest gate, queried by individual generators (and by `solveOneNode`) before they bother to run. Decoded from `whenCuts_` and the node depth. CLI: `-cuts on/root/ifmove/forceOn/off` maps onto the `whenCuts_` state machine (`CbcModel.cpp:10142-10198`, with the magic state values 999999, 999998, 5000010, 8000008, 10000004, 10000006).

Most influential parameters (existing CLI)

Parameter	Default	Why it matters in the tree
<code>-cutPasses</code>	adaptive (50/100/-100)	root pass cap; controls how much of the per-generator decay actually fires
<code>-passCuts</code>	4 (adaptive)	per-node pass cap; lower = cheaper nodes, less bound improvement deep in the tree
<code>-treeCutDepth</code>	6	hard cut-off depth; <i>the</i> master switch for cuts in deep nodes
<code>-cuts</code>	on	decodes into <code>whenCuts_</code> (drives <code>doCutsNow</code>)
<code>-cutDepth</code>	-1 (off)	also try cuts at depths 0, N , $2N$, ... regardless of <code>treeCutDepth</code>
<code>-slowcutpasses</code>	10	cap on “slow” cut passes
<code>-multipleRootSolversoff</code>		paces several LP/cut configs at root, deposits all into <code>globalCuts_</code>
<code>-conflictCuts</code>	on	toggles dive conflict-cut harvest (§9, item 6)

Hardcoded constants worth promoting to CLI

These influence behaviour just as much as the parameters above but are baked into the source. Listed in descending order of expected tuning impact:

Constant	Source line	What it controls
Global-pool violation cutoff 0.005	<code>CbcModel.cpp:906</code>	Filter for re-scan; a tighter value (e.g. 10^{-3}) would let the tree benefit more from earlier root cuts. Candidate CLI: <code>-globalCutMinViolation</code> .
Global-pool element budget $\max(nz/10, 2n)+100$	<code>CbcModel.cpp:907</code>	The single most influential “how aggressively can the LP grow” knob. Candidate CLI: <code>-globalCutBudget</code> .
<code>CUT_HISTORY = 7</code>	<code>CbcModel.cpp:838</code>	Length of the rolling buffer used by <code>ifMove</code> . Shorter = exit cut loop sooner. Candidate CLI: <code>-cutHistory</code> .
<code>currentPassNumber_ ≥ 10</code> guard	<code>CbcModel.cpp:965</code>	Suppresses early termination for the first 10 passes. Candidate CLI: <code>-cutMinPasses</code> .
Small-model size $≤ 500$	<code>CbcModel.cpp:1839</code>	Engers the simpler <code>doCutsNow</code> branch. Candidate CLI: <code>-cutSmallModelSize</code> .
<code>TRY_IDEA1=2</code> depth gate (= 10)	<code>CbcModel.cpp:1893</code>	Where <code>allowForTopOfTree=3</code> fires. Candidate CLI: <code>-cutDepthGate</code> .
<code>SCANCUTS = 1000</code>	<code>CbcCutGenerator.cpp:110</code>	Generator scan cap. Candidate CLI: <code>-cutFullScanEvery</code> .
Slack-cut re-admit $100 \tau_{\text{prim}}$	<code>CbcModel.cpp:1170</code>	Tolerance for re-injecting parked cuts.

Constant	Source line	What it controls
Dive conflict cap 128	CbcRootHeuristicSchedule.cpp:26	Depth of dive conflict cuts; already settable from the API but no CLI flag. Candidate CLI: <code>-diveConflictMaxCuts</code> .
<code>drop[12]</code> {1, 2, 3, 10, ..., 1000}	table CbcModel.cpp:967	Per-depth bonus added to the <code>ifMove</code> early-termination test.

12 The branchAndBound entry point

`CbcModel::branchAndBound()` (`CbcModel.cpp:2700+`) is the top-level driver of the entire MIP solve. Its responsibilities, in order:

1. Initial LP solve and feasibility check.
2. Presolve / preprocessing (if enabled).
3. **Root heuristics:** call `doHeuristicsAtRoot()` (§13).
4. **Root cut loop:** `solveWithCuts(cuts, maximumCutPassesAtRoot_, NULL)` (§2).
5. **Post-root generator adjustment:** the `howOften` decay pass (§4).
6. **Tree search:** the main `while` loop pops nodes, applies cuts, branches, and pushes children (§15).

It owns the `CbcModel` state that all inner routines share: `solver_` (the `OsiClp` LP), `globalCuts_`, `generator_[]`, `tree_`, and all pass counters.

13 doHeuristicsAtRoot: pre-cut primal heuristics

`CbcModel::doHeuristicsAtRoot()` (`CbcModel.cpp:3056`) runs early primal heuristics *before* the root cut loop, so that cut generators can benefit from a tighter incumbent bound:

1. **Feasibility Pump** (FPump): rounds the LP relaxation toward an integer-feasible solution.
2. **Dive heuristics with conflict harvesting** (the MIPster extension via `CbcRootHeuristicSchedule`): multiple quick dive heuristics run in sequence; when a dive proves infeasible at a node, the resulting conflict cut is collected.
3. Conflict cuts are scored by normalised violation $((\text{lhs} - \text{ub})/\text{nz})$, the top `conflictMaxCuts_ = 128` are applied to the LP (§9, item 6), and any globally valid ones are deposited into `globalCuts_`.

Any incumbent found here immediately tightens the cutoff, which prunes the cut loop and the subsequent tree search.

14 applyRowCuts + resolve: cut installation

After a batch of cuts is produced by the generators, they are installed into the LP and the relaxation is re-solved. This two-step operation lives at `CbcModel.cpp:9514-9533`:

```
solver_ -> applyRowCuts(numberCuts, cuts); // add rows
solver_ -> resolve();                     // re-optimize LP
```

`applyRowCuts` delegates to `OsiClpSolverInterface::applyRowCuts`, which in turn calls `ClpModel::addRows` — appending rows to the LP constraint matrix in bulk (no resolve yet). The subsequent

`resolve()` uses the existing warm-start basis and typically converges in very few simplex iterations because the new rows are unlikely to change many basic variables.

Side effects:

- `whichGenerator_[]` is populated with the generator index for each new row (§17).
- `numberNewCuts_` is incremented.
- If the resolve proves infeasible, the pass loop exits immediately (the subproblem is fathomed).

15 Node selection and branching

Inside the tree-search loop of `branchAndBound` (`CbcModel.cpp:~17600+`), each iteration:

1. **Pop:** the tree strategy (`CbcTree`, typically best-first on bound) selects the next live node.
2. **Restore:** `addCuts(node, lastws)` rebuilds the LP for that node (§18).
3. **Branch:** the branching decision (recorded in the node's `CbcBranchingObject`) is applied as a bound change on the branching variable. The LP is resolved with the tightened bound.
4. **Cut generation:** `solveWithCuts(cuts, maximumCutPasses_, node)` runs 1–4 passes (§11).
5. **Store:** new cuts are wrapped in `CbcCountRowCut` and owned by the node's `CbcNodeInfo` (§17).
6. **Push children:** child nodes are created and pushed to the tree; each inherited cut's refcount is incremented by the number of children.

If the node is fathomed (infeasible, or bound $\geq$ incumbent), steps 4–6 are skipped and all inherited cuts are decremented.

16 Method and data-structure maps

Two diagrams below show the cut orchestration: one for the **root node** (where most cuts are generated) and one for **tree nodes** (where cuts are restored and selectively regenerated). Blue boxes are methods, orange boxes are data structures, green boxes are parameters/constants that gate transitions.

Diagram 1: Root-node cut orchestration

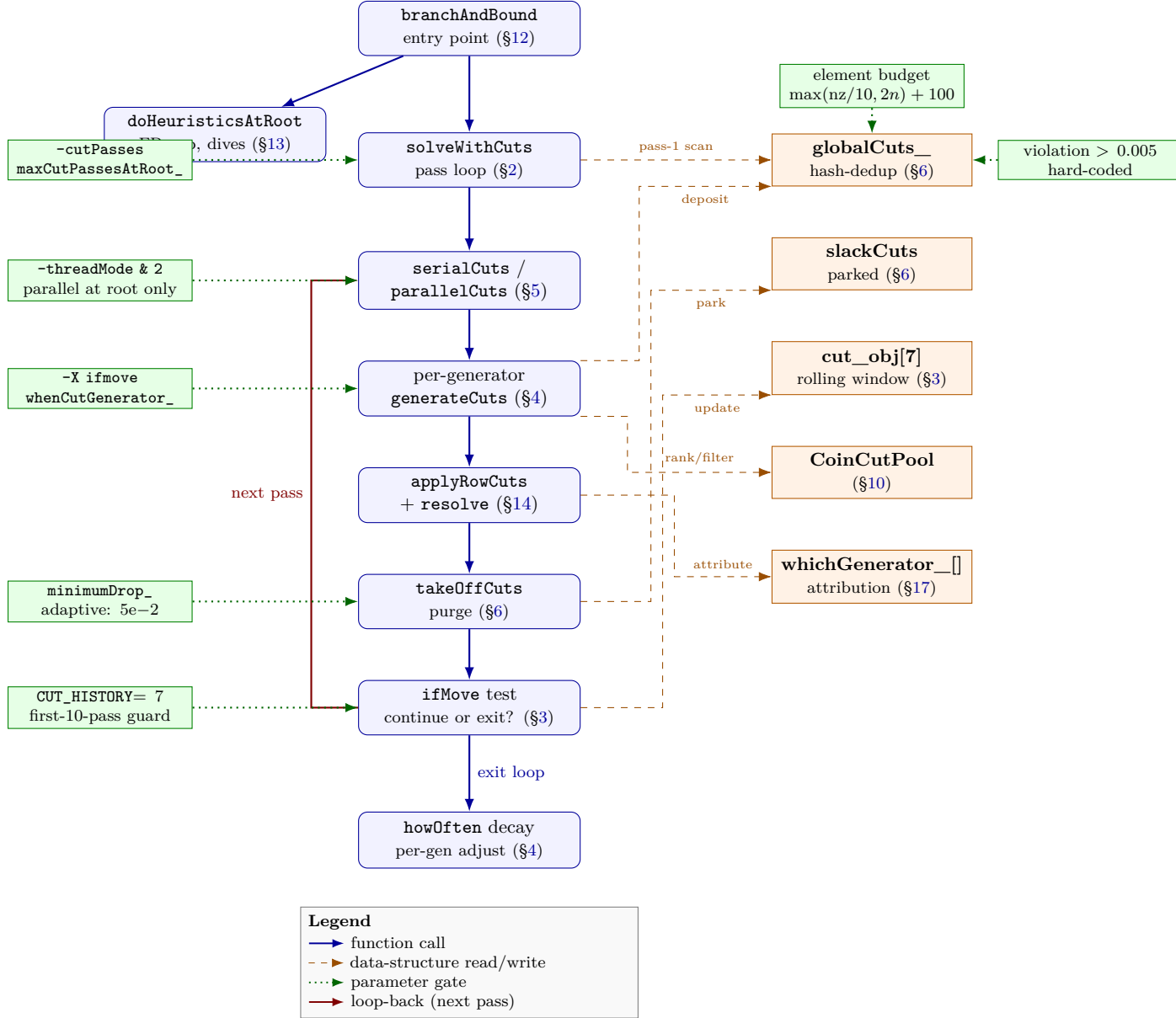
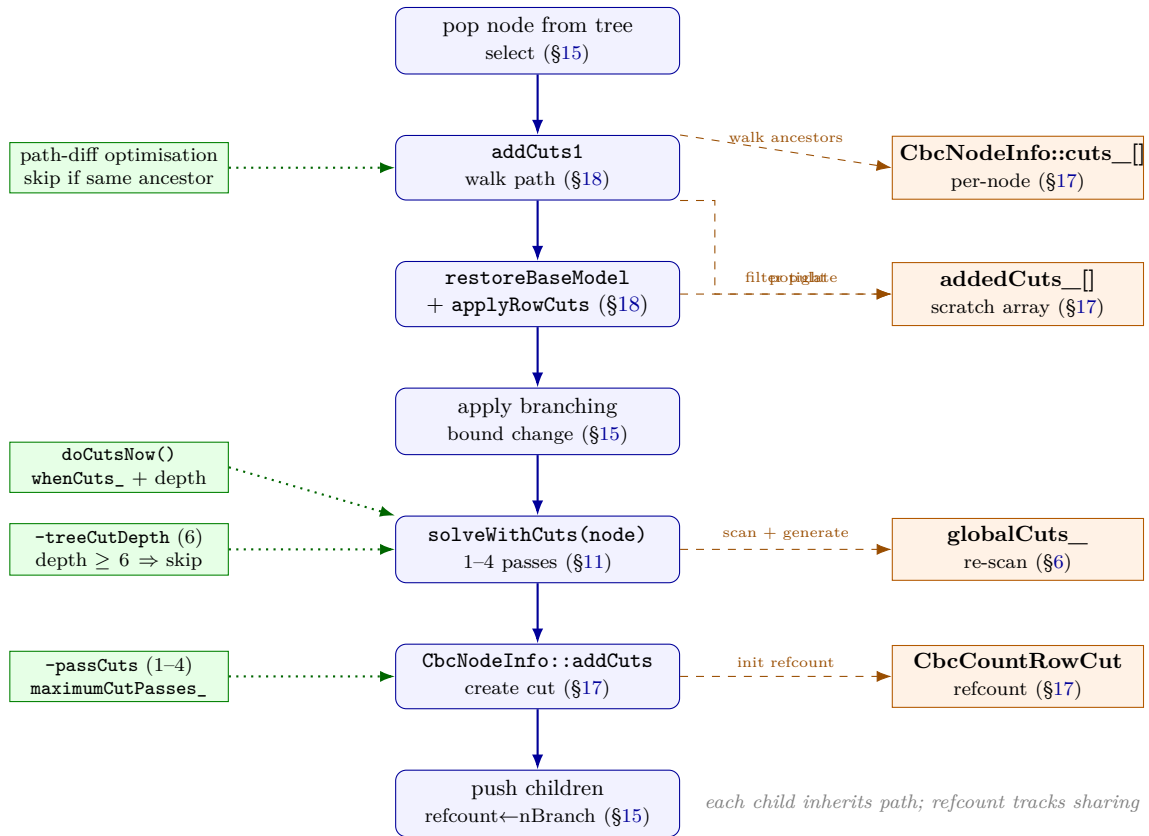


Diagram 2: Tree-node cut lifecycle



How to read the diagrams. **Diagram 1** (root) shows the pass loop inside `solveWithCuts`: generators fire in batch, cuts are applied and resolved, `takeOffCuts` purges non-binding ones, and the `ifMove` test decides whether to loop or exit. Parameters on the left gate each step; data structures on the right are written/read at the indicated points. The red loop-back arrow shows the per-pass iteration.

Diagram 2 (tree) shows the lifecycle at each non-root node: the LP is reconstructed wholesale from the ancestor path (`addCuts1` → `restoreBaseModel`), branching is applied, then `solveWithCuts` runs 1–4 passes (gated by `treeCutDepth` and `doCutsNow`). New cuts become `CbcCountRowCut` objects owned by the node’s `CbcNodeInfo`, with `recount` initialised to the number of child branches.

17 Tree-side cut data structures and lifecycle

Understanding how cuts survive beyond the root and are reconstructed at each tree node requires knowledge of five interacting data structures.

1. `addedCuts_` — the per-node scratch array

```

// CbcModel.hpp:3146
CbcCountRowCut **addedCuts_;
int maximumNumberCuts_; // allocated capacity
int currentNumberCuts_; // entries currently valid
  
```

`addedCuts_` is a **temporary array rebuilt each time a node is selected from the tree**. It collects pointers to every `CbcCountRowCut` on the root→node path. It does *not* own the cuts — the actual owners are the `CbcNodeInfo` objects (see below).

Population: `addCuts1(node, lastws)` at `CbcModel.cpp:7969` walks the tree path, calling `applyToModel()` on each ancestor; each call appends that ancestor's `cuts_[]` entries into `addedCuts_[offset...]`.

Filtering: after population, code at line 8169 classifies each entry as *tight* (non-basic in the warm-start) or *loose* (basic). Loose entries are set to NULL (and reference-counted down). Only tight cuts are installed in the LP.

Cleanup during cut generation: `takeOffCuts` (line 11341) decrements and possibly frees entries that become slack after a resolve.

Fathoming: when a node is pruned by bound (line 8326–8337), *all* entries are decremented by `numberBranchesLeft` and freed if `refcount` reaches zero.

2. CbcCountRowCut — reference-counted cut

```
// CbcCountRowCut.hpp
class CbcCountRowCut : public OsiRowCut {
    CbcNodeInfo *owner_;           // back-pointer to owning node
    int ownerCut_;                 // index in owner's cuts_[]
    int numberPointingToThis_;     // reference count
    int whichCutGenerator_;
};
```

Reference counting protocol:

- **Creation:** initialised to the number of child branches that will inherit it (typically 2).
- **Decrement sites:** (a) node fathomed by bound (all its inherited cuts lose `numberBranchesLeft` refs), (b) `takeOffCuts` removes a slack ancestor cut (−1), (c) tree cleanup on node deletion (−`numberLeft`).
- **At zero:** the destructor calls `owner_>deleteCut(ownerCut_)` to null the slot in the owning `CbcNodeInfo`'s `cuts_[]` array.

`canDropCut()` (line 97–112 of `CbcCountRowCut.cpp`) returns `false` for cuts with `effectiveness()==COIN_DBL_` (never droppable) and for `effectiveness()>1e10` cuts that are currently tight.

3. CbcNodeInfo — per-node persistent storage

```
// CbcNodeInfo.hpp
class CbcNodeInfo {
    CbcCountRowCut **cuts_;        // cuts generated AT this node
    int numberCuts_;               // length of cuts_[]
    int numberRows_;              // rows before these cuts
    CbcNodeInfo *parent_;         // chain to root
};
```

Each `CbcNodeInfo` stores *only* the cuts generated at that specific node. The full cut set is implicitly the union along the root→node path, reconstructed lazily by `addCuts1()`.

`CbcNodeInfo::addCuts(OsiCuts&, numberToBranch, numberPointingToThis)` at `CbcNodeInfo.cpp:326` allocates `CbcCountRowCut` objects, sets `owner_=this` and the initial `refcount` to `numberToBranch`. It is called after `solveWithCuts` returns new cuts (line 16737).

`CbcPartialNodeInfo` (the common case) additionally stores a *basis diff* and bound changes applied at this node.

4. whichGenerator_[] — cut attribution

```
// CbcModel.hpp:3427
int *whichGenerator_; // sized maximumWhich_ (initial 1000)
```

Maps active cut positions (from `numberRowsAtContinuous_`) to generators. Encoding:

Value	Meaning
<code>i + 10000</code>	root/global cut from generator <i>i</i>
<code>i + 20000</code>	tree-generated cut from generator <i>i</i>
<code>20099</code>	cut from <code>globalCuts_</code> pool (unattributed)
<code>20098</code>	stored/reinstated global cut

Used by the post-loop decay logic to count how many cuts each generator has active, then adjust `howOften` adaptively.

5. Row layout in the solver

At any node, the LP row structure is:

```
[0 .. numberRowsAtContinuous_-1] = original LP rows
[numberRowsAtContinuous_ .. + numberOldActiveCuts_-1] = ancestor cuts
[.. + numberNewCuts_-1] = this node's cuts
```

`numberOldActiveCuts_` = ancestor cuts still tight after `addCuts1` filtering. `numberNewCuts_` = cuts generated in `solveWithCuts` at this node.

18 Node cut restoration: the wholesale rebuild

Unlike some MIP solvers that incrementally add/remove cuts when moving between nodes, MIPster uses a **wholesale rebuild**:

1. **Walk root→node.** `addCuts1()` walks the parent chain via `walkback_[]`, collecting all `CbcNodeInfo*` and summing their cut counts.
2. **Apply path.** Traverses `walkback_[]` from root to node. Each `CbcPartialNodeInfo::applyToModel()` applies: (a) basis diff, (b) bound changes from branching, (c) copies its `cuts_[]` pointers into `addedCuts_[]`.
3. **Filter tight vs. loose.** Each cut's warm-start basis status is inspected: non-basic $\Rightarrow$ tight (keep), basic $\Rightarrow$ loose (drop, decrement refcount). Exception: `effectiveness() > 1e10` cuts that `canDropCut()` refuses.

4. **Strip + reinstall.**

```
solver_ -> restoreBaseModel(numberRowsAtContinuous_);
solver_ -> applyRowCuts(numberToAdd, addedCuts);
solver_ -> setWarmStart(lastws);
```

5. **Optimisation: path diff.** Lines 8016–8068 compare the new path with `lastNodeInfo_[]` (previous node's path). If paths share ancestors, only the divergent suffix needs processing. If identical (`sameProblem=true`), the whole rebuild is skipped.

Why wholesale, not incremental? The warm-start basis and the tight/loose classification change across nodes. Re-evaluating every ancestor cut against the new basis is cheaper than tracking individual insertions/deletions — particularly since `Osiclp::restoreBaseModel` is an $O(1)$ pointer reset (it stores a cached matrix at the continuous).

19 fullScan and periodic generator re-evaluation

The `fullScan` variable in `serialCuts` (`CbcModel.cpp:9112-9118`) controls whether all generators run or only frequent ones:

```
int fullScan = 0;
if ((numberNodes_ % SCANCUTS) == 0 || (specialOptions_ & 256) != 0) {
    fullScan = 1;
    if (!numberNodes_ || (specialOptions_ & 256) != 0)
        fullScan = 2;
}
```

fullScan	Meaning
0	Normal: only generators with active <code>howOften</code> run; those with <code>switchOff</code> are skipped.
1	Periodic (every <code>SCANCUTS</code> =1000 nodes): all generators fire to re-assess which should remain active.
2	Root (<code>numberNodes</code> =0) or new-solution trigger (<code>specialOptions_&256</code>): most aggressive scan;

The companion variable `switchOff` (line 11003) is:

```
int switchOff = (!doCutsNow(1) && !fullScan) ? 1 : 0;
```

When `switchOff`=1, generators with default frequency are silently skipped — this is how the depth-based `doCutsNow()` gating actually suppresses generators in the tree without modifying their `howOften`.

20 moreSpecialOptions_ and specialOptions_ cut-related bits

`moreSpecialOptions_` (`CbcModel.hpp:3209`)

Bits	Mask	Effect on cuts
11–12	»11 &3	<code>experimentBreak</code> : 0=old style early-exit, 1=new style, 2=no break if zero cuts first time, 3=as 2 but last drop must be > 0.1·min.
13	8192	<code>increaseDrop</code> : ramp up <code>minimumDrop_</code> across passes (bail out faster).
16	65536	No cuts during preprocessing.
20	1048576	Resolve LP <i>before</i> cut generation starts (reduce sum of infeasibilities).
21	2097152	Resolve LP <i>after</i> cuts at root.

`specialOptions_` (`CbcModel.hpp:3178`)

Bit	Value	Effect on cuts
0	1	<code>debugCuts</code> mode (<code>OsiRowCutDebugger</code> active).
8	256	New solution found $\Rightarrow$ trigger full generator scan at next node (<code>fullScan</code> =2).
11	2048	Small sub-B&B: skip <code>treeCutDepth_</code> check.
12	4096	“Funny cuts, do slow way” (affects <code>restoreBaseModel</code> path).
13	8192	Same as 4096 in alternative code paths.

Bit	Value	Effect on cuts
23	8388608	Leave cuts in solver after solve (don't strip).

21 LP reoptimisation cost control — design notes

Status: not yet implemented. This section records the design rationale so the work can be picked up later.

Problem statement

On some instances, the LP reoptimisation that follows cut insertion takes an unexpectedly large number of simplex iterations, making the root cut loop extremely slow. The causes are:

1. **Row bloat** — too many cuts add too many rows; each pivot operates on a larger basis.
2. **Dense cuts** — high NZ-per-row cuts increase the per-pivot work.
3. **Already-large LP** — when the LP is already large, adding even a moderate number of rows compounds the cost.

Currently, the *only* NZ budget in the codebase is the global pool re-scan cap at `CbcModel.cpp:9342`:

$$\text{maximumAdd} = \max(\text{nz}/10, 2n + 100)$$

Generator-produced cuts (all passes) have **no such cap**. Every cut surviving `filterCutsByBestPerVar` is applied unconditionally at line 9815. This is the gap.

The right signal: simplex iterations per cut pass

`solver_>getIterationCount()` computed as a *delta* across the post-cut-insertion `resolve()` is the correct deterministic proxy for LP difficulty. It is:

- **Deterministic** — a function of LP state only, not wall time.
- **Self-calibrating** — naturally proportional to LP size and conditioning.
- **Already partially present** — `cutIterations` at `CbcModel.cpp:9880` captures the post-resolve count, but as a total rather than a per-pass delta.

The capture pattern needed:

```
int itersBefore = solver_>getIterationCount();
applyRowCuts(...);
resolve();                                     // the expensive call
int itersThisPass = solver_>getIterationCount() - itersBefore;
takeOffCuts(...);                             // may also resolve, but not counted here
```

Two complementary levers

Lever 1 — per-pass NZ budget for generator cuts (preventive)

Extend the same budget logic used for the global pool to generator-produced cuts. After `filterCutsByBestPerVar` and before building the `addCuts[]` array (around `CbcModel.cpp:9800`):

```
CoinBigIndex nzBudget = std::max(
    (CoinBigIndex)(cutPassNZRatio_ * solver_>getNumElements()),
    (CoinBigIndex)(2 * solver_>getNumCols() + 100));
// sort theseCuts by fitness descending; drop cuts exceeding budget
```


This mirrors the global pool formula with a configurable ratio. Two parameters:

- **-cutPassNZRatio** *R* (float, default ∞ = current unlimited behaviour). Suggested tuning range: 0.2–0.5. The global pool uses $R \approx 0.1$; a higher value here is justified because generator cuts are already pre-filtered by `filterCutsByBestPerVar`.
- **-cutPassMaxNZ** *N* (int, default -1 = disabled). Absolute NZ ceiling per pass, regardless of LP size. Combined with the ratio as $\min(\text{ratio budget}, N)$. Useful when the ratio-based budget is too generous on very large LPs.

Why it does not hurt easy instances. On small/sparse LPs the floor $2n + 100$ dominates and the budget exceeds the total NZ of all generated cuts. The cap only bites when the LP is already large and adding dense cuts would make it much larger.

Lever 2 — LP iteration count gate (reactive)

After measuring `itersThisPass`, stop further cut generation if the LP became too hard:

```
if (cutPassMaxIterations_ > 0
    && itersThisPass > cutPassMaxIterations_) {
    numberTries = 0;    // exit cut loop; B&B continues normally
}
```

Parameter: **-cutPassMaxIterations** *N* (int, default -1 = disabled). A natural initial benchmark: $N = 10 \times n_{\text{cols}}$, meaning “if the LP needed more than 10 pivots per variable to re-solve, stop generating cuts”.

Why it does not hurt easy instances. The gate never fires if LP reoptimisation is cheap. The threshold expressed as a multiple of column count is automatically proportional to problem size.

How the two levers compose

Scenario	Lever 1 effect	Lever 2 effect
Small LP, sparse cuts, few iterations	Floor dominates $\Rightarrow$ all cuts admitted	Threshold not hit $\Rightarrow$ all passes run
Large LP, many dense cuts, LP still fast	Budget caps NZ $\Rightarrow$ fewer cuts per pass	Threshold not hit $\Rightarrow$ passes continue
Large LP, LP became hard (many iterations)	Budget already restricting NZ growth	Gate fires $\Rightarrow$ cut loop exits

Lever 1 is **preventive** (avoids making the LP hard). Lever 2 is **reactive** (stops adding cuts once the LP is already hard). They cover different failure modes and can be used independently or together.

One design subtlety

Setting `numberTries = 0` when the iteration gate fires stops the cut loop entirely. A softer variant would be to skip the next pass but allow the loop to retry after a few passes (“cooldown”). This would protect against the case where the LP is temporarily hard after a particular cut but would benefit from further cuts once the dual solution settles. The implementation would keep a `cutPassCooldown` counter rather than setting `numberTries = 0`.

What to leave out

- **Time-based fallbacks** — explicitly excluded for determinism.

- **Per-generator NZ caps** — already served by `-cutLength` (Gomory) and the `switchOffIfLessThan_` mechanism; no need to duplicate at this level.
- **Un-nooping the `densityNew > 200` block** (`CbcModel.cpp:10579–10594`) — that block is post-loop and only affects future *tree* nodes, not the current root pass. It is a separate, complementary mechanism worth revisiting independently.

Implementation scope

Both levers touch a small, well-defined region of `solveWithCuts`:

- **Lever 1:** ~25 lines after `CbcModel.cpp:9657` (after `filterCutsByBestPerVar`), before line 9815 (`applyRowCuts`). Sort `theseCuts` by fitness (reusing `CoinCutPool::rowCutFitness`), then trim to NZ budget.
- **Lever 2:** ~8 lines after the `resolve()` at line 9834. Capture `itersBefore` just before `applyRowCuts`, compute delta, check threshold.

Plus the standard boilerplate for each parameter: field + getter/setter in `CbcModel.hpp`; init in constructor, copy, and assign (5 sites); entry in `CbcParam.hpp`; `CbcParameters.cpp` registration; wire-up in `CbcSolver.cpp`.

22 Complete parameter catalog

Parameter	Default	Purpose
<code>-cutPasses</code>	100	Root pass cap (overridden adaptively to 50/100/−100 based on column count).
<code>-passCuts</code>	1	Per-tree-node pass cap (maps to <code>maximumCutPasses_</code> ; solver uses 4 adaptively).
<code>-treeCutDepth</code>	6	Hard depth cutoff for cuts in tree.
<code>-cutDepth</code>	−1	Per-generator depth modulus (<code>depthCutGenerator_</code>).
<code>-cuts</code>	on	Master <code>whenCuts_</code> control (on/root/ifmove/forceOn/off).
<code>-slowcutpasses</code>	10	Cap on calls to slow generators (RedSplit, LandP).
<code>-cutLength</code>	−1	Gomory cut density cap.
<code>-conflictCuts</code>	on	Dive conflict-cut harvest.
<code>-multipleRootSolvers</code>	off	Race LP/cut configs at root.
<code>-threadMode</code>	0	Bit 2: parallel root cuts.
<code>-cliqueCuts</code>	ifmove	BKClique generator mode.
<code>-flowCoverCuts</code>	ifmove	Flow cover generator mode.
<code>-gomoryCuts</code>	ifmove	Gomory generator mode.
<code>-knapsackCuts</code>	ifmove	Knapsack cover generator mode.
<code>-mixedIntegerRoundingCuts</code>	ifmove	MIR generator mode.
<code>-oddwheelCuts</code>	off	Odd-wheel generator mode.
<code>-probingCuts</code>	ifmove	Probing generator mode.
<code>-twoMirCuts</code>	ifmove	TwoMIR generator mode.
<code>-zeroHalfCuts</code>	ifmove	ZeroHalf generator mode.
<code>-reduceAndSplitCuts</code>	off	RedSplit generator mode.
<code>-reduce2AndSplitCuts</code>	ifmove	RedSplit2 generator mode.
<code>-GMICuts</code>	off	GMI generator mode.
<code>-liftAndProjectCuts</code>	off	LandP generator mode.

Parameter	Default	Purpose
-lagomoryCuts	off	Lagomory generator mode.
-latwomirCuts	off	LaTwoMIR generator mode.
-residualCapacityCuts	off	ResidualCapacity generator mode.
-pathAggrCuts	root	Path aggregation generator mode.
-zeroHalfSparseThreshold	8000	ZeroHalf: switch to sparse graph.
-zeroHalfRowMaxPairCount	150000	ZeroHalf: skip dense rows.
-cutRankingMetric	fitness	Scoring formula at all 3 ranking sites: violation (raw LP violation) or fitness $((viol/activeCols) \cdot 10^5 + (1/(\Delta_C + 1)) \cdot 10^3)$.
-cutPoolFilter	on	Enable per-round best-per-variable filter (filterCutsByBestPerVar).
-cutPoolFilterMinCuts	50	Minimum row cuts per round to activate the filter; rounds with fewer cuts are unfiltered.
-globalCutMinViolation	0.005	Minimum violation for a global-pool cut to be re-admitted to the LP.

B. Per-generator internal defaults (`installCutGenerators`)

Generator	Key limits set	Accuracy	switchOffIfLessThan
Probing	maxPass=1, maxProbe=123, maxLook=10/20, maxElements=200/300, rowCuts=-3	5	0
Gomory	limitAtRoot=1000, limit=50, awayAtRoot=0.005	3	0
Knapsack	(no special limits)	1	0
MIR/Flow	(default params)	2	0
PathAggr	(no special limits)	2	0
TwoMIR	maxElements=250, awayAtRoot=0.005, away=0.01	4	1 (disabled pass 0 if 0 cuts)
Clique	(CoinCutPool-based selection inside)	0	0
OddWheel	(CoinCutPool-based selection inside)	0	0
ZeroHalf	sparse threshold 8000, pair-count 150000	5	2 (disabled pass 0 if <2 cuts)
RedSplit/LandP	maximumTries=slowcutpasses (10)	5	0 (uses maximumTries_ instead)

C. Hardcoded constants (candidates for CLI promotion)

Constant	Value	Source	What it controls
Global-pool budget	element $\max(nz/10, 2n) + 100$	CbcModel.cpp:934	Maximum NZ the LP can grow per global-pool injection.
CUT_HISTORY	7	CbcModel.cpp:854	Rolling-window length for ifMove early-exit.

Constant	Value	Source	What it controls
First-pass guard	≥ 10	CbcModel.cpp:965	Suppress early termination for first 10 passes.
Slack re-admit cutoff	$100 \tau_{\text{prim}}$	CbcModel.cpp:1110	Tolerance for re-injecting parked cuts.
Small-model threshold	≤ 500	CbcModel.cpp:1839	Triggers simpler <code>doCutsNow</code> branch.
TRY_IDEA1 depth gate	$= 10$	CbcModel.cpp:1893	Where <code>allowForTopOfTree=3</code> fires.
SCANCUTS	1000	CbcCutGenerator.cpp:608	Nodes between full generator scans.
Dive conflict cap	128	CbcRootHeuristic.cpp:42	Cap for dive conflict cuts.
Density abandon thresholds	$1e-5$ obj, 20% nz	CbcModel.cpp:~10270	Abandon tree cuts if root cuts didn't help but bloated the LP.
<code>drop[12]</code> table	$\{1, 2, 3, 10, \dots, 1000\}$	CbcModel.cpp:967	Per-depth bonus on ifMove early-termination.
<code>effectiveness > 1e10</code>	10^{10}	CbcModel.cpp:817	Threshold for “always keep” cuts (below <code>COIN_DBL_MAX</code>).

23 Putting it together

A typical root iteration looks like this:

1. LP solve $\Rightarrow$ fractional solution x^* .
2. Pre-cut heuristics may produce an incumbent.
3. Pass 1 of `solveWithCuts`:
 - Scan `globalCuts_` for violations of x^* , sort by violation, add up to a matrix-element budget.
 - Run all enabled generators (serial, or parallel if `threadMode_&2`).
 - Apply, resolve.
 - `takeOffCuts`: drop slack cuts whose effectiveness is finite.
 - Update `cut_obj` history; check `badObj` / `goodDrop`.
4. Pass $k > 1$: as above plus a primal-heuristic block on the new x^* before the cut generation.
5. Loop ends when (a) the pass cap is hit, or (b) the `ifMove` test (§3) fires while no generator is in `mustCallAgain` mode.
6. Post-loop adjustment: every generator's `howOften` is rewritten based on its yield; ineffective ones drop to -99 .
7. Branch. Inside the tree, `solveWithCuts` runs again with `maximumCutPasses_` (default 4 in MIPster), no global-pool re-scan beyond pass 1, no parallel cuts, and deeper-than-`treeCutDepth_` nodes skip cuts entirely.

A typical **tree node** iteration:

1. Pop node from tree. Call `addCuts(node, lastws)`:
 - `addCuts1`: walk root $\rightarrow$ node, rebuild `addedCuts_[]`, classify tight vs. loose.

- `restoreBaseModel` + `applyRowCuts` (tight only) + `setWarmStart`.
2. Apply branching decision (bound change).
 3. LP resolve. If infeasible or fathomed by bound, prune (decrement refcounts on inherited cuts, done).
 4. `solveWithCuts(cuts, maximumCutPasses_, node)`:
 - `treeCutDepth_` gate: $\text{depth} \geq 6 \Rightarrow \text{skip}$.
 - `doCutsNow()` gate: depth/modular check.
 - If allowed: run `serialCuts`, apply, resolve, `takeOffCuts` — typically only 1 pass.
 5. Store results: `CbcNodeInfo::addCuts()` creates `CbcCountRowCut` objects for new cuts with initial refcount = number of child branches.
 6. Push child nodes to tree.

File index

File	Role
<code>src/CbcModel.cpp</code>	root/tree orchestration, cut loop, take-off, pool scan, <code>addCuts1</code>
<code>src/CbcModel.hpp</code>	<code>maximumCutPasses[AtRoot]_</code> , <code>globalCuts_</code> , <code>addedCuts_</code> , <code>whichGenerat</code>
<code>src/CbcCutGenerator.cpp/hpp</code>	per-generator gating, switches bitfield, <code>SCANCUTS</code>
<code>src/CbcCountRowCut.cpp/hpp</code>	<code>CbcRowCuts</code> pool, dedup; <code>CbcCountRowCut</code> refcounting
<code>src/CbcNodeInfo.hpp</code>	per-node cut storage (<code>cuts_[]</code> , <code>numberCuts_</code>), tree walk helpers
<code>src/CbcPartialNodeInfo.cpp</code>	<code>applyToModel</code> : basis diff + bounds + cut population
<code>src/CbcThread.cpp</code>	<code>parallelCuts</code> , threading mode bits
<code>src/CbcSolver.cpp</code>	adaptive defaults, generator/heuristic registration
<code>src/CbcSolverCutSetup.cpp</code>	<code>installCutGenerators</code> : per-generator limits
<code>src/CbcParameters.cpp</code>	CLI parameter setup, defaults
<code>src/CbcTree.cpp</code>	tree cleanup, refcount decrement on deleted nodes

Negative results / failed experiments

Preprocessing probing row-cut filter (2026-06)

Hypothesis: When `CglProbing` generates a large number of row cuts during preprocessing passes (observed: 890 K cuts on `rocII-5-11`, similar explosions on `supportcase10` and `cdma`), filtering them through a fitness-ranked pool (`CoinCutPool`, score = viol/activeCols, threshold max(5·rows, 10 000)) before LP re-insertion would reduce LP NZ growth and therefore shorten preprocessing.

Result: no meaningful improvement. A 17-instance × 2-condition (filter / no-filter) experiment on MIPLIB-2017+SPP instances (`probing_filter_20260628_093943`) showed:

- Preprocessing times differed by < 15% between conditions, well within noise.
- Nodes explored were essentially identical (0 for most instances).
- The worst-case instances (`supportcase10`, `cdma`) timed out in both conditions.

Root cause: The bottleneck is *cut generation time*, not LP solve time. `CglProbing` can spend thousands of seconds generating cuts before the filter even activates. The filter only acts on the resulting `OsiCuts` object; it does not shorten the generation itself. Additionally, for instances

like `usafa`, the bottleneck is the initial LP relaxation (3600 s) before preprocessing begins — a separate issue.

Follow-up direction: The correct fix is a time limit *inside* `CglProbing::generateCutsAndModify` so that the generator itself aborts when preprocessing wall-clock budget is nearly exhausted.